Mixed Integer Programming

Sensitivity: Internal

Contents

- Review –MILP
- Binary Variables
  - Knapsack Example
  - Set Covering Problem Example
  - Purchasing Problem

Sensitivity: Internal

Review

Sensitivity: Internal

What is a Mixed-Integer Linear Program

- Linear Objective
- Linear Constraints
- Some or all of the decision variables are restricted to be integers

e.g

maximise $P = 2x + 3.5y$

subject to:

$$3x + 2y \leq 14$$
$$2x + 3y \leq 12.5$$
$$-x + y \leq 1$$

and

$$x, y \geq 0$$
$$x, y \in \mathbb{Z}$$

maximise $P = 2x + 3.5y$

subject to:

$$3x + 2y \leq 14$$
$$2x + 3y \leq 12.5$$
$$-x + y \leq 1$$

and

$$x, y \geq 0$$
$$x \in \mathbb{Z}$$

Sensitivity: Internal

Linear Program Solution Without Integers

maximise $P = 2x + 3.5y$

subject to:

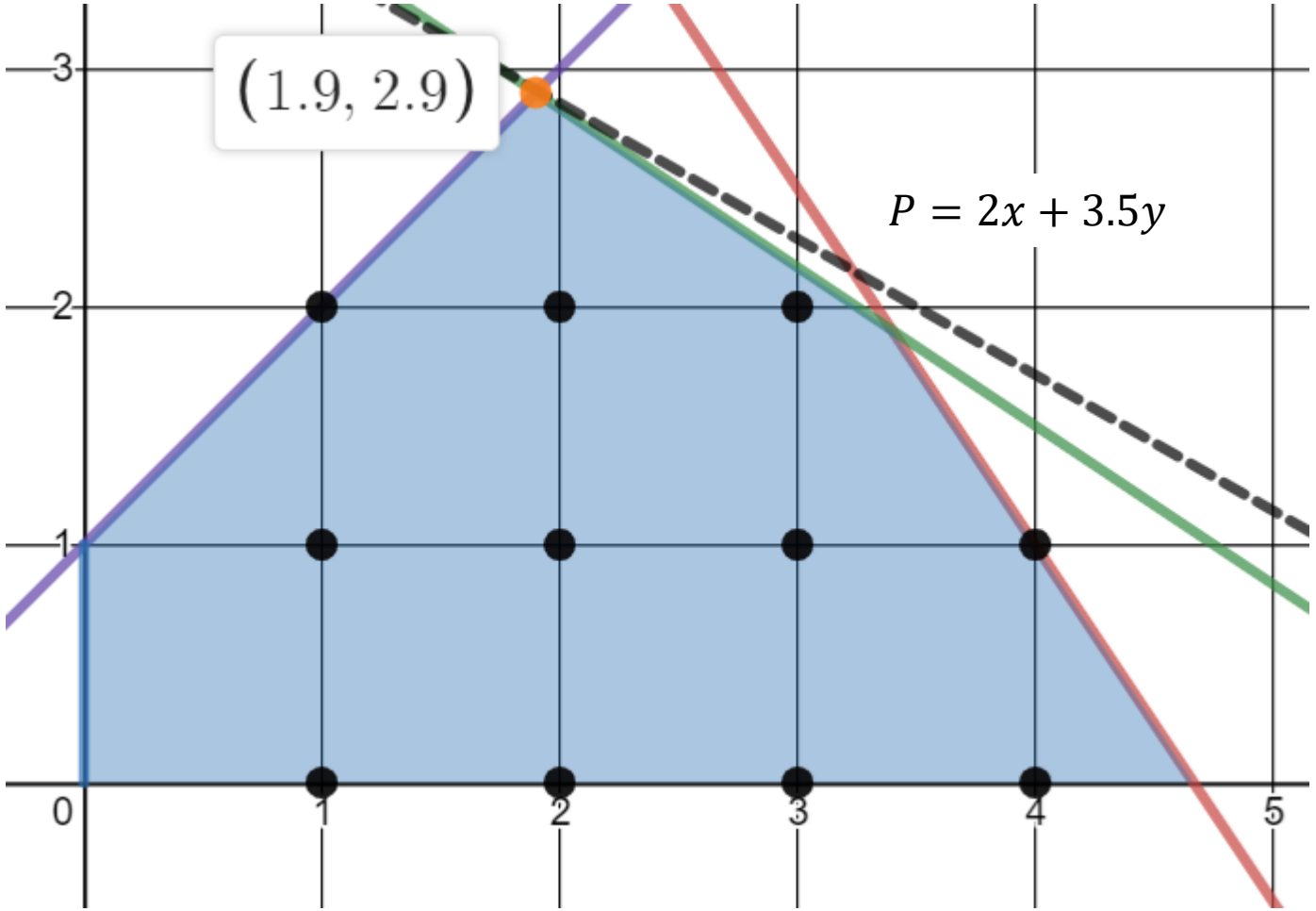
$$3x + 2y \leq 14$$
$$2x + 3y \leq 12.5$$
$$-x + y \leq 1$$

and

$$x, y \geq 0$$

Optimal Solution

$$\boldsymbol{x}^* = (x, y) = (1.9, 2.9)$$
$$P^* = 13.95$$



Sensitivity: Internal

Rounded Solutions

Optimal Solution

$$\boldsymbol{x}^* = (x, y) = (1.9, 2.9)$$
$$P^* = 13.95$$

$$\boldsymbol{x} = (x, y) = (1,2)$$

$$P = 9$$

$$\boldsymbol{x} = (x, y) = (1,3)$$

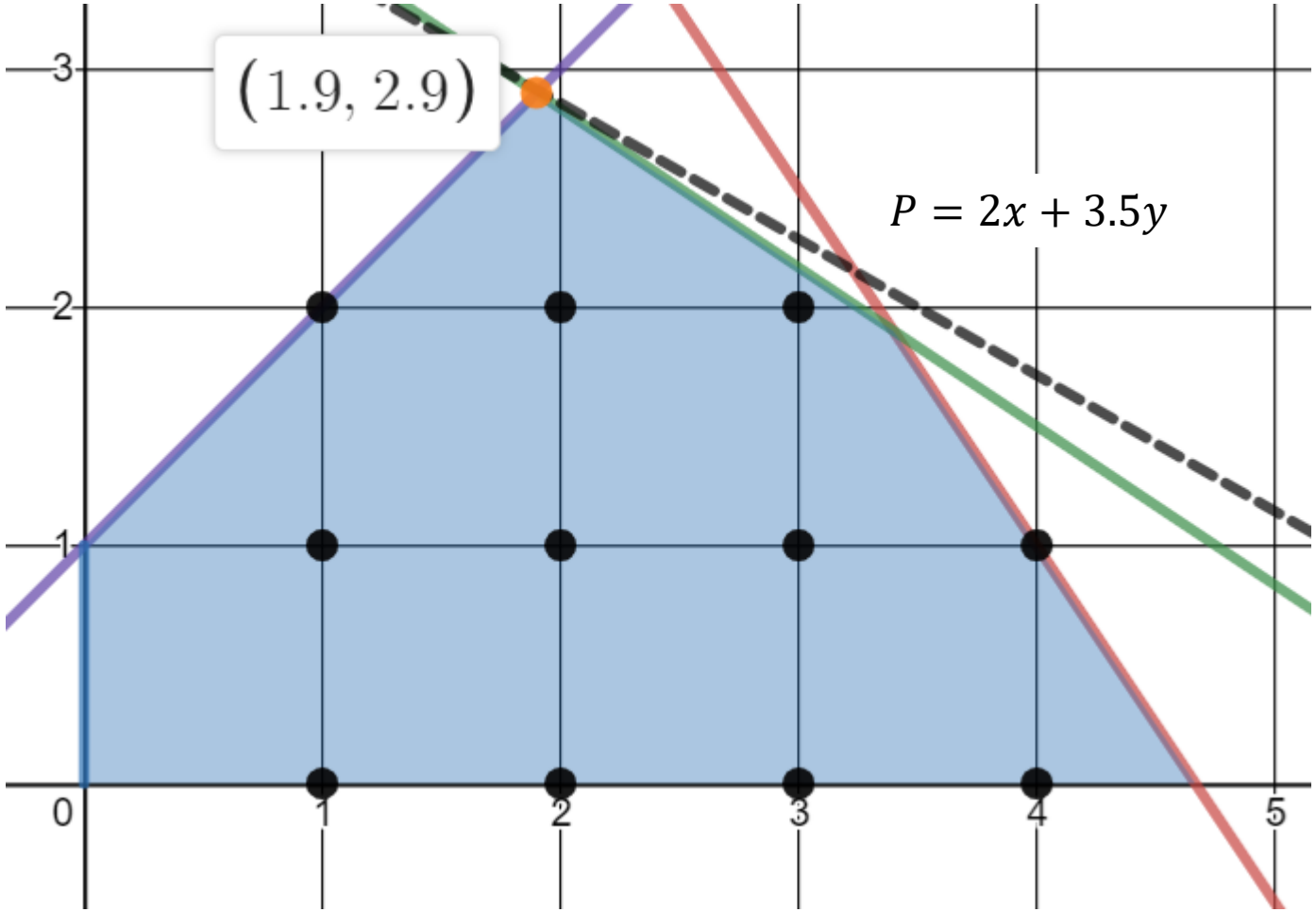
Infeasible!

$$\boldsymbol{x} = (x, y) = (2,2)$$

$$P = 11$$

$$\boldsymbol{x} = (x, y) = (2,3)$$

Infeasible!



Sensitivity: Internal

Rounded Solutions

$$\boldsymbol{x} = (x, y) = (1,2)$$

$$P = 9$$

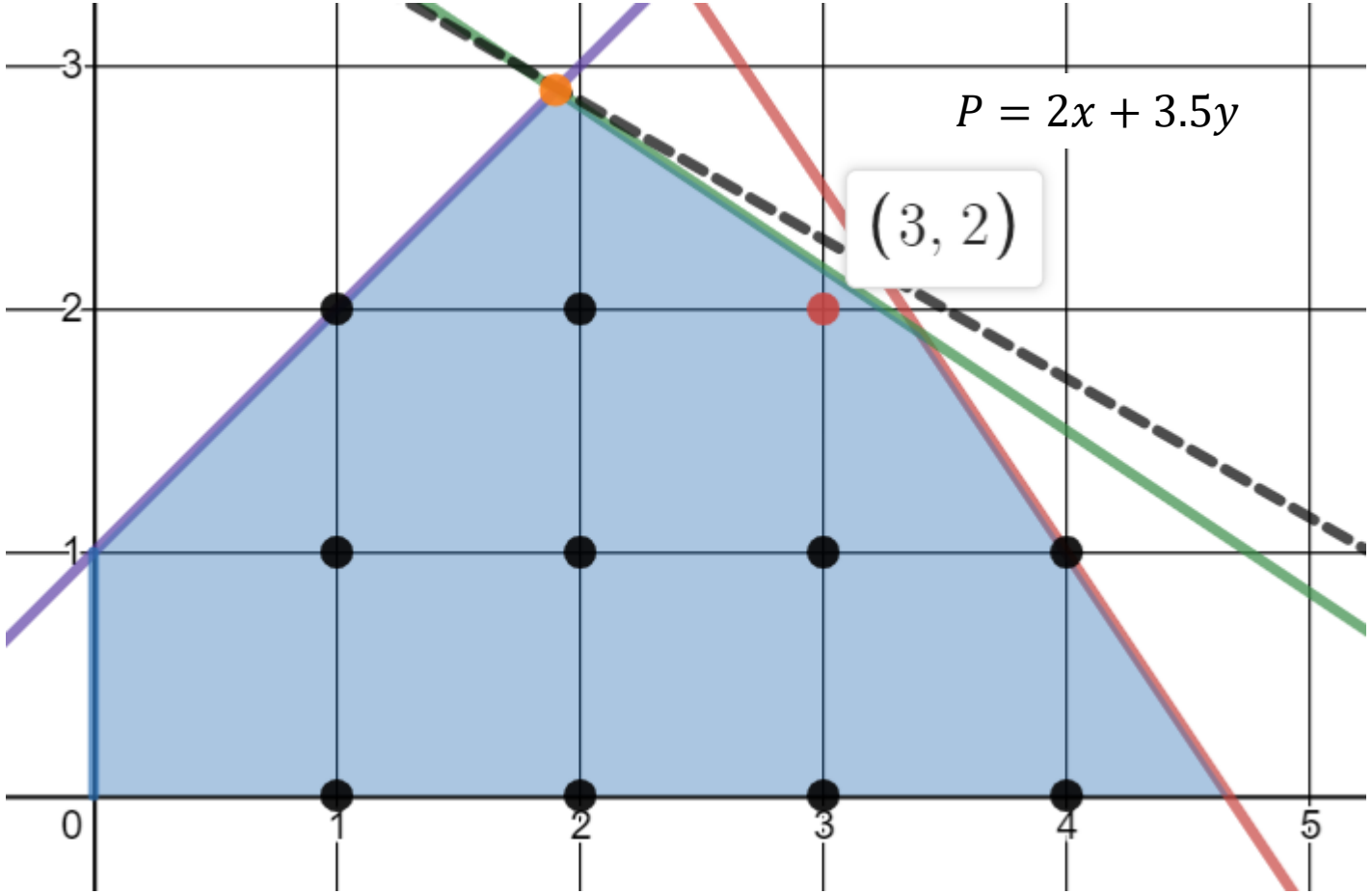
$$\boldsymbol{x} = (x, y) = (2,2)$$

$$P = 11$$

However!

$$\boldsymbol{x}^* = (x, y) = (3,2)$$

$$P^* = 13$$



Sensitivity: Internal

Integer Solution via Pyomo

maximise $P = 2x + 3.5y$

subject to:

$$3x + 2y \leq 14$$
$$2x + 3y \leq 12.5$$
$$-x + y \leq 1$$

and

$$x, y \geq 0$$
$$x, y \in \mathbb{Z}$$

Optimal Solution

$\boldsymbol{x}^* = (x, y) = (3,2)$
$P^* = 13$

```python
from pyomo.environ import *

# Create a concrete model
model = ConcreteModel()

# Define the integer decision variables with lower and upper bounds
model.x = Var([0, 1], domain=NonNegativeIntegers)

# Define the objective function coefficients as a list
f = [2, 3.5]

# Define the objective function
model.obj = Objective(expr=sum(f[i] * model.x[i] for i in [0, 1]), sense=maximize)

# Add the constraints to the model
model.c1 = Constraint(expr=3 * model.x[0] + 2 * model.x[1] <= 14)
model.c2 = Constraint(expr=2 * model.x[0] + 3 * model.x[1] <= 12.5)
model.c3 = Constraint(expr=-model.x[0] + model.x[1] <= 1)

# Create a solver
solver = SolverFactory('glpk')

# Display the model
model.pprint()

# Solve the model
solver.solve(model, tee=False)  # Set tee=True to display solver output

# Display the results
model.display()
```

```
sol =

      3
      2


fval =

    -13
```

Sensitivity: Internal

Binary Variables

Sensitivity: Internal

What is a Binary Variable

- A decision variable that is either 0 or 1

maximise $P = 2x + 3.5y$

subject to:

$$3x + 2y \leq 14$$
$$2x + 3y \leq 12.5$$
$$-x + y \leq 1$$

and

$$x, y \geq 0$$
$$x \in \{0,1\}$$

maximise $P = 2x + 3.5y$

subject to:

$$3x + 2y \leq 14$$
$$2x + 3y \leq 12.5$$
$$-x + y \leq 1$$

and

$$x, y \geq 0$$
$$x \leq 1$$
$$x \in \mathbb{Z}$$

This is also an integer variable restricted to ≥ 0 and ≤ 1

Sensitivity: Internal

Knapsack Problem

- 7 items, can only carry 10kg, which to pick to maximise value?





Sensitivity: Internal

Knapsack Problem

For n items and a capacity W

maximise

$$\sum_{i=1}^{n} v_i x_i$$

subject to:

$$\sum_{i=1}^{n} w_i x_i \leq \mathrm{W}$$

$$x_i \in \{0,1\}, \quad \forall i$$

Item - x_i	Weight (kg) - w_i	Value (£) - v_i
1	2	10
2	2	10
3	2	10
4	6	100
5	6	100
6	8	130
7	3	70

For our example $n = 7$ and $\mathrm{W} = 10$

Sensitivity: Internal

Knapsack Problem via Pyomo

Maximise

$$\sum_{i=1}^{7} v_i x_i$$

subject to:

$$\sum_{i=1}^{7} w_i x_i \leq 10$$

$$x_i \in \{0,1\}, \quad \forall i$$

Item - x_i	Weight (kg) - w_i	Value (£) - v_i
1	2	10
2	2	10
3	2	10
4	6	100
5	6	100
6	8	130
7	3	70

```python
from pyomo.environ import *

# Create a concrete model
model = ConcreteModel()

# Define the binary variables
model.x = Var(range(7), within=Binary)

# Define objective function values and weights
value = [10, 10, 10, 100, 100, 130, 70]
weight = [2, 2, 2, 6, 6, 8, 3]
W = 10

# Define the objective function
model.obj = Objective(expr=sum(value[i] * model.x[i] for i in range(7)), sense=maximize)

# Define the capacity constraint
model.capacity = Constraint(expr=sum(weight[i] * model.x[i] for i in range(7)) <= W)

# Create a solver
solver = SolverFactory('glpk')
# Display the model
model.pprint()
# Solve the model
solver.solve(model)
# Display the results
model.display()
```

Sensitivity: Internal

Set Covering Problem - Fire Stations Example

- There are six cities in region R. The region must determine where to build fire stations.
- The region wants to build the minimum number of fire stations and ensure that at least one fire station is within 15 minutes of each city.

The times (in minutes) required to drive between cities are:

City	1	2	3	4	5	6
1	0	10	20	30	30	20
2	10	0	25	35	20	10
3	20	25	0	15	30	20
4	30	35	15	0	15	25
5	30	20	30	15	0	14
6	20	10	20	25	14	0

Sensitivity: Internal

Set Covering Problem - Fire Stations Example

This is a connected network!

We are removing edges that don't allow us to reach another node in 15 minutes

City	1	2	3	4	5	6
1	0	10	20	30	30	20
2	10	0	25	35	20	10
3	20	25	0	15	30	20
4	30	35	15	0	15	25
5	30	20	30	15	0	14
6	20	10	20	25	14	0

By inspection, build for city 2 and 4.

Covers 1,2,6 and 3,4,5.

Sensitivity: Internal

Set Covering Problem - Fire Stations Example

City	1	2	3	4	5	6
1	0	10	20	30	30	20
2	10	0	25	35	20	10
3	20	25	0	15	30	20
4	30	35	15	0	15	25
5	30	20	30	15	0	14
6	20	10	20	25	14	0

Let

$$x_i = \begin{cases} 1 - \text{if a fire station is built} \\ 0 - \text{otherwise} \end{cases}$$

Objective?

Minimise the number of fire stations built!

minimise

$$\sum_{i=1}^{6} x_i = x_1 + x_2 + x_3 + x_4 + x_5 + x_6$$

Sensitivity: Internal

Set Covering Problem - Fire Stations Example

City	1	2	3	4	5	6
1	0	10	20	30	30	20
2	10	0	25	35	20	10
3	20	25	0	15	30	20
4	30	35	15	0	15	25
5	30	20	30	15	0	14
6	20	10	20	25	14	0

Constraints?

Consider City 1, who can reach it in 15 minutes?

- City 1 and City 2

$x_1 + x_2 \geq 1$ (City 1 Constraint)

Consider City 2, who can reach it in 15 minutes?

- City 1, City 2 and City 6

$x_1 + x_2 + x_6 \geq 1$ (City 2 Constraint)

Sensitivity: Internal

Set Covering Problem - Fire Stations Example

City	1	2	3	4	5	6
1	0	10	20	30	30	20
2	10	0	25	35	20	10
3	20	25	0	15	30	20
4	30	35	15	0	15	25
5	30	20	30	15	0	14
6	20	10	20	25	14	0

minimise

$$\sum_{i=1}^{6} x_i = x_1 + x_2 + x_3 + x_4 + x_5 + x_6$$

subject to:

$$x_1 + x_2 \geq 1 \qquad \text{(City 1 Constraint)}$$

$$x_1 + x_2 + x_6 \geq 1 \qquad \text{(City 2 Constraint)}$$

$$x_3 + x_4 \geq 1 \qquad \text{(City 3 Constraint)}$$

$$x_3 + x_4 + x_5 \geq 1 \qquad \text{(City 4 Constraint)}$$

$$x_4 + x_5 + x_6 \geq 1 \qquad \text{(City 5 Constraint)}$$

$$x_2 + x_5 + x_6 \geq 1 \qquad \text{(City 6 Constraint)}$$

Sensitivity: Internal

Set Covering Problem - Fire Stations Example

minimise

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6$$

subject to:

$$\begin{aligned}
x_1 + x_2 &\geq 1 \\
x_1 + x_2 + x_6 &\geq 1 \\
x_3 + x_4 &\geq 1 \\
x_3 + x_4 + x_5 &\geq 1 \\
x_4 + x_5 + x_6 &\geq 1 \\
x_2 + x_5 + x_6 &\geq 1
\end{aligned}$$

And

$$x_i \in \{0,1\}, \qquad \forall i$$

Sensitivity: Internal

Set Covering Problem - Fire Stations Example

minimise

$$\sum_{i=1}^{6} x_i = x_1 + x_2 + x_3 + x_4 + x_5 + x_6$$

subject to:

$x_1 + x_2 \geq 1$ (City 1 Constraint)

$x_1 + x_2 + x_6 \geq 1$ (City 2 Constraint)

$x_3 + x_4 \geq 1$ (City 3 Constraint)

$x_3 + x_4 + x_5 \geq 1$ (City 4 Constraint)

$x_4 + x_5 + x_6 \geq 1$ (City 5 Constraint)

$x_2 + x_5 + x_6 \geq 1$ (City 6 Constraint)

and

$$x_i \in \{0,1\}, \qquad \forall i$$

```python
from pyomo.environ import *

# Create a concrete model
model = ConcreteModel()

# Define the binary variables
model.x = Var(range(6), within=Binary)

# Define the objective function
model.obj = Objective(expr=sum(model.x[i] for i in range(6)), sense=minimize)

# Define the inequality constraints
model.city1 = Constraint(expr=model.x[0] + model.x[1] >= 1)
model.city2 = Constraint(expr=model.x[0] + model.x[1] + model.x[5] >= 1)
model.city3 = Constraint(expr=model.x[2] + model.x[3] >= 1)
model.city4 = Constraint(expr=model.x[2] + model.x[3] + model.x[4] >= 1)
model.city5 = Constraint(expr=model.x[3] + model.x[4] + model.x[5] >= 1)
model.city6 = Constraint(expr=model.x[1] + model.x[4] + model.x[5] >= 1)

# Create a solver
solver = SolverFactory('glpk')
# Display the model
model.pprint()
# Solve the model
solver.solve(model)
# Display the results
model.display()
```

Sensitivity: Internal

Purchasing Problem

Item	Cost (per kg)	Expected Profit (per kg)
1	10	5
2	12	3
3	9	4
4	5	3
5	13	3

Maximise the expected profit given that you have a budget of £160 and can order a maximum of 10 per item.

In addition, add the following:

- Minimum order placed must be at least 6
  - You can order at most 2 different items
  - You can order either item 1 or item 4

Sensitivity: Internal

Basic Model

Maximum order of 10 per item

Maximise

$$\sum_{i=1}^{5} P_i x_i$$

subject to:

$$\sum_{i=1}^{5} c_i x_i \leq 160$$

$$x_i \geq 0, \qquad \forall i$$
$$x_i \leq 10, \qquad \forall i$$

Item	Cost (per kg)	Expected Profit (per kg)
1	10	5
2	12	3
3	9	4
4	5	3
5	13	3

P_i - Profit (per kg) for item i

c_i - Cost (per kg) for item i

Sensitivity: Internal

Updated Problem

Item	Cost (per kg)	Expected Profit (per kg)
1	10	5
2	12	3
3	9	4
4	5	3
5	13	3

Minimum order per item is at least 6

Maximise

$$\sum_{i=1}^{5} P_i x_i$$

subject to:

$$\sum_{i=1}^{5} c_i x_i \leq 160$$

$$x_i \geq 6, \qquad \forall i$$
$$x_i \leq 10, \qquad \forall i$$

What is wrong with this?

Answer: I don't have to order an item!

This requires at least 6 of every item to be chosen.

I want to say the following.

IF I choose to order an Item **THEN**

I must order at least 6 and no more than 10

Sensitivity: Internal

Binary Variables!

Minimum order per item is at least 6

Maximise

$$\sum_{i=1}^{5} P_i x_i$$

subject to:

$$\sum_{i=1}^{5} c_i x_i \leq 160$$

$$x_i \leq 10\delta_i, \qquad \forall i \quad (1)$$
$$x_i \geq 6\delta_i, \qquad \forall i \quad (2)$$

and

$$\delta_i \in \{0,1\}, \qquad \forall i$$

Item	Cost (per kg)	Expected Profit (per kg)
1	10	5
2	12	3
3	9	4
4	5	3
5	13	3

δ_i – 1 if item i picked, 0 if item i not picked

When $\delta_i = 0$, constraints (1) and (2) become:

$$x_i \leq 0$$
$$x_i \geq 0$$

i.e. You **don't** choose item i – force lower and upper bounds of 0

When $\delta_i = 1$, constraints (1) and (2) become:

$$x_i \leq 10$$
$$x_i \geq 6$$

i.e. You **do** choose item i – force lower and upper bounds of $6 \leq x_i \leq 10$

Sensitivity: Internal

In Pyomo

Minimum order per item is at least 6

Maximise

$$\sum_{i=1}^{5} P_i x_i$$

subject to:

$$\sum_{i=1}^{5} c_i x_i \leq 160$$

$$x_i \leq 10\delta_i, \qquad \forall i$$
$$x_i \geq 6\delta_i, \qquad \forall i$$

and

$$\delta_i \in \{0,1\}, \qquad \forall i$$

```python
from pyomo.environ import *

# Define cost and profit as lists
cost = [10, 12, 9, 5, 13]
profit = [5, 3, 4, 3, 3]

# Create a concrete model
model = ConcreteModel()

# Define the decision variables and binary variables
model.x = Var(range(5), within=NonNegativeReals)
model.b = Var(range(5), within=Binary)

# Define the objective function
model.obj = Objective(expr=sum(profit[i] * model.x[i] for i in range(5)), sense=maximize)

# Define the constraints
model.cost_max = Constraint(expr=sum(cost[i] * model.x[i] for i in range(5)) <= 160)
model.select = ConstraintList()
model.lower_order = ConstraintList()

for i in range(5):
    model.select.add(model.x[i] <= 10 * model.b[i])
    model.lower_order.add(model.x[i] >= 6 * model.b[i])

# Create a solver
solver = SolverFactory('glpk')
# Display the model
model.pprint()
# Solve the model
solver.solve(model, tee=True)
# Display the results
model.display()
```

Sensitivity: Internal

Additional Constraint

You can order amounts of at most 2 items

maximise

$$\sum_{i=1}^{5} P_i x_i$$

subject to:

$$\sum_{i=1}^{5} c_i x_i \leq 160$$

$$x_i \leq 10\delta_i, \qquad \forall i$$
$$x_i \geq 6\delta_i, \qquad \forall i$$

$$\sum_{i=1}^{5} \delta_i \leq 2$$

and

$$\delta_i \in \{0,1\}, \qquad \forall i$$

Item	Cost (per kg)	Expected Profit (per kg)
1	10	5
2	12	3
3	9	4
4	5	3
5	13	3

Sensitivity: Internal

In Pyomo

You can order amounts of at most 2 items

Maximise

$$\sum_{i=1}^{5} P_i x_i$$

subject to:

$$\sum_{i=1}^{5} c_i x_i \leq 160$$

$$x_i \leq 10\delta_i, \qquad \forall i$$
$$x_i \geq 6\delta_i, \qquad \forall i$$

$$\sum_{i=1}^{5} \delta_i \leq 2$$

and

$$\delta_i \in \{0,1\}, \qquad \forall i$$

```python
from pyomo.environ import *

# Define cost and profit as lists
cost = [10, 12, 9, 5, 13]
profit = [5, 3, 4, 3, 3]

# Create a concrete model
model = ConcreteModel()

# Define the decision variables and binary variables
model.x = Var(range(5), within=NonNegativeReals)
model.b = Var(range(5), within=Binary)

# Define the objective function
model.obj = Objective(expr=sum(profit[i] * model.x[i] for i in range(5)), sense=maximize)

# Define the constraints
model.cost_max = Constraint(expr=sum(cost[i] * model.x[i] for i in range(5)) <= 160)
model.select = ConstraintList()
model.lower_order = ConstraintList()
model.no_to_order = Constraint(expr=sum(model.b[i] for i in range(5)) <= 2)

for i in range(5):
    model.select.add(model.x[i] <= 10 * model.b[i])
    model.lower_order.add(model.x[i] >= 6 * model.b[i])

# Create a solver
solver = SolverFactory('glpk')
# Display the model
model.pprint()
# Solve the model
solver.solve(model)
# Display the results
model.display()
```

Sensitivity: Internal

Purchasing Problem

Item	Cost (per kg)	Expected Profit (per kg)
1	10	5
2	12	3
3	9	4
4	5	3
5	13	3

You can only order either item 1 or item 4, not both

maximise

$$\sum_{i=1}^{5} P_i x_i$$

subject to:

$$\sum_{i=1}^{5} c_i x_i \leq 160$$

$$x_i \leq 10\delta_i, \qquad \forall i$$
$$x_i \geq 6\delta_i, \qquad \forall i$$

$$\delta_1 + \delta_4 \leq 1$$

and

$$\delta_i \in \{0,1\}, \qquad \forall i$$

Sensitivity: Internal

Purchasing Problem

You can only order either item 1 or item 4, not both

Maximise

$$\sum_{i=1}^{5} P_i x_i$$

subject to:

$$\sum_{i=1}^{5} c_i x_i \leq 160$$

$$x_i \leq 10\delta_i, \qquad \forall i$$
$$x_i \geq 6\delta_i, \qquad \forall i$$

$$\delta_1 + \delta_4 \leq 1$$

and

$$\delta_i \in \{0,1\}, \qquad \forall i$$

```python
from pyomo.environ import *

# Define cost and profit as lists
cost = [10, 12, 9, 5, 13]
profit = [5, 3, 4, 3, 3]

# Create a concrete model
model = ConcreteModel()

# Define the decision variables and binary variables
model.x = Var(range(5), within=NonNegativeReals)
model.b = Var(range(5), within=Binary)

# Define the objective function
model.obj = Objective(expr=sum(profit[i] * model.x[i] for i in range(5)), sense=maximize)

# Define the constraints
model.cost_max = Constraint(expr=sum(cost[i] * model.x[i] for i in range(5)) <= 160)
model.select = ConstraintList()
model.lower_order = ConstraintList()
model.item1_or_item4 = Constraint(expr=model.b[0] + model.b[3] <= 1)

for i in range(5):
    model.select.add(model.x[i] <= 10 * model.b[i])
    model.lower_order.add(model.x[i] >= 6 * model.b[i])

# Create a solver
solver = SolverFactory('glpk')
# Display the model
model.pprint()
# Solve the model
solver.solve(model)
# Display the results
model.display()
```

Sensitivity: Internal